

AI ASSISTED CODING LAB ASSIGNMENT =18.1

Name: Pothunuri Akshitha

Roll no:2403A510C2

Batch no:05

CSE 2nd year

AI-Assisted Coding

Task - 01

Prompt:

Generate and automatically run a Python script (under 50 lines) using the requests library that connects to a public weather API (such as OpenWeatherMap) to fetch current weather details for a given city.

The script should:

- Prompt the user to enter a city name.
- Use an API key for authentication.
- Handle errors for invalid city name, invalid/missing API key, and network issues.
- Display temperature (°C), humidity (%), and weather description in a readable format.
- Include try-except blocks for error handling.
- Keep the code clean, simple, and under 50 lines.
- Automatically execute the code after generation.

Code generated :

```
weather.py > ...
1  import os, sys, requests
2
3
4  def fetch_weather(city, api_key):
5      try:
6          r = requests.get(
7              "https://api.openweathermap.org/data/2.5/weather",
8              params={"q": city, "appid": api_key, "units": "metric"},
9              timeout=10,
10          )
11          if r.status_code == 401:
12              raise ValueError("Invalid or missing API key.")
13          if r.status_code == 404:
14              raise ValueError("City not found. Check the city name.")
15          r.raise_for_status()
16          return r.json()
17      except requests.exceptions.RequestException as e:
18          raise ConnectionError(f"Network error: {e}")
19
20
21 def prompt_or_arg(prompt_text, fallback):
22     if fallback:
23         return fallback
24     if sys.stdin and sys.stdin.isatty():
25         try:
26             return input(prompt_text).strip()
27         except EOFError:
28             return None
29     return None
30
31
32 def main():
```

```
weather.py > ...
32 def main():
33     city_arg = next((a for a in sys.argv[1:] if not a.startswith("--")), None)
34     api_arg = None
35     for a in sys.argv[1:]:
36         if a.startswith("--api-key="):
37             api_arg = a.split("=", 1)[1]
38     city = prompt_or_arg("Enter city name: ", city_arg)
39     if not city:
40         print("City name is required (pass as arg or run interactively).")
41         sys.exit(1)
42     api_key = os.getenv("OPENWEATHER_API_KEY") or prompt_or_arg(
43         "Enter OpenWeatherMap API key: ", api_arg
44     )
45     if not api_key:
46         print(
47             "API key is required (env OPENWEATHER_API_KEY, --api-key, or interactive).")
48         )
49         sys.exit(1)
50     try:
51         data = fetch_weather(city, api_key)
52         desc = data["weather"][0]["description"].title()
53         temp_c = data["main"]["temp"]
54         humidity = data["main"]["humidity"]
55         print(f"Weather in {city}: {desc}")
56         print(f"Temperature: {temp_c:.1f} °C")
57         print(f"Humidity: {humidity}%")
58     except (ValueError, ConnectionError) as e:
59         print(f"Error: {e}")
60         sys.exit(1)
61
62
```

```
requirements.txt flask_api.py student.py weather.py x app.py
weather.py > ...
61
62
63 if __name__ == "__main__":
64     main()
65
```

Output:

```
C:\Users\nadhi\OneDrive\Desktop\AI>python weather.py
Enter city name: Warangal
Enter OpenWeatherMap API key: 34
Error: Invalid or missing API key.

C:\Users\nadhi\OneDrive\Desktop\AI>
```

Observation:

The code efficiently fetches and displays real-time weather details with robust error handling for invalid input, API issues, and network failures.

It's well-structured, user-friendly, and keeps execution flexible via command-line arguments or interactive prompts.

Task – 02

Prompt:

Generate and automatically run a Python script (under 50 lines) that connects to a public currency exchange API (like exchangerate-api.com or Frankfurter API) to convert INR to USD, EUR, and GBP.

The script should:

- Use the requests library for the API call.
- Ask the user to enter an amount in INR.
- Validate that the input amount is numeric and positive.
- Handle errors for invalid API responses or network downtime.
- Display the conversion results for USD, EUR, and GBP in a neat tabular format (using string formatting, not external libraries).
- Include proper try-except error handling.
- Keep the code simple, readable, and under 50 lines.
- Automatically execute after generation.

Code generated:

currency_converter.py > ...

```
1  import sys
2  import requests
3
4  API_URL = "https://api.frankfurter.app/latest"
5  TARGETS = ["USD", "EUR", "GBP"]
6
7  def read_positive_amount() -> float:
8      raw = input("Enter amount in INR: ").strip()
9      try:
10         amount = float(raw)
11         if amount <= 0:
12             raise ValueError("Amount must be positive")
13         return amount
14     except ValueError:
15         print("Invalid input. Please enter a positive number.")
16         sys.exit(1)
17
18  def fetch_rates(amount_in_inr: float) -> dict:
19      try:
20         params = {"amount": amount_in_inr, "from": "INR", "to": ",".join(TARGETS)}
21         resp = requests.get(API_URL, params=params, timeout=10)
22         resp.raise_for_status()
23         data = resp.json()
24         if "rates" not in data or not isinstance(data["rates"], dict):
25             raise ValueError("Malformed API response")
26         return data["rates"]
27     except (requests.RequestException, ValueError) as exc:
28         print(f"Error fetching rates: {exc}")
29         sys.exit(2)
30
31  def print_table(amount_in_inr: float, rates: dict) -> None:
32      print("\nCurrency Conversion (INR -> USD, EUR, GBP)")
33      print("-" * 44)
34      print(f"{'From':<10}{'To':<10}{'Rate':>12}{'Amount':>12}")
35      print("-" * 44)
```

currency_converter.py > ...

```
31 def print_table(amount_in_inr: float, rates: dict) -> None:
32     print("-" * 44)
33     for code in TARGETS:
34         value = rates.get(code)
35         if value is None:
36             continue
37         print(f"{'INR':<10}{'code':<10}{value/amount_in_inr:>12.6f}{value:>12.2f}")
38     print("-" * 44)
39
40 def main() -> None:
41     amount = read_positive_amount()
42     rates = fetch_rates(amount)
43     print_table(amount, rates)
44
45 if __name__ == "__main__":
46     main()
47
48
49
50
51
```

Output:

```
Error: Invalid or missing API key.

C:\Users\nadhi\OneDrive\Desktop\AI>python currency_converter.py
Enter amount in INR: 500

Currency Conversion (INR -> USD, EUR, GBP)
-----
From      To      Rate      Amount
-----
INR       USD      0.011271    5.64
INR       EUR      0.009755    4.88
INR       GBP      0.008600    4.30
-----

-----
INR       USD      0.011271    5.64
INR       EUR      0.009755    4.88
-----

-----
INR       USD      0.011271    5.64
INR       EUR      0.009755    4.88
-----

-----
INR       USD      0.011271    5.64
INR       EUR      0.009755    4.88
-----

-----
INR       USD      0.011271    5.64
-----

-----
INR       USD      0.011271    5.64
INR       EUR      0.009755    4.88
INR       GBP      0.008600    4.30
-----

C:\Users\nadhi\OneDrive\Desktop\AI>
```

Observation:

The code cleanly converts INR to USD, EUR, and GBP using a live API with strong input validation and error handling.

It's efficient, readable, and presents results in a clear, well-formatted table.

Task – 03

Prompt:

Generate and automatically run a **Python script (under 50 lines)** that connects to the **GitHub REST API** to fetch repository details (name, description, stars, forks, and open issues).

Requirements:

- Use the **requests** library and API endpoint: `https://api.github.com/repos/{owner}/{repo}`
- Prompt the user to enter the repository name (e.g.,

NadhiyaSeelam/AI_ASSISTED_CODING_2.1).

- If the repository is private, allow the user to provide a **GitHub Personal Access Token** (via input) and include it in request headers as:

```
headers = {"Authorization": "token YOUR_GITHUB_TOKEN"}
```

Code generated:

```
github_repo_info.py > ...
1  import requests
2
3  def fetch_repo_info():
4      repo_name = input("Enter repository name (owner/repo, e.g., NadhiyaSeelam/AI_ASSISTED_CODING_2.1): ").strip()
5      url = f"https://api.github.com/repos/{repo_name}"
6
7      print("\nAttempting to fetch repository info...")
8      try:
9          response = requests.get(url)
10
11         if response.status_code == 404:
12             print("Repository not found. Please check the name and try again.")
13             return
14
15         if response.status_code == 401:
16             print("This repository is private or requires authentication.")
17             token = input("Enter your GitHub Personal Access Token (or press Enter to skip): ").strip()
18             if token:
19                 headers = {"Authorization": f"token {token}"}
20                 response = requests.get(url, headers=headers)
21             else:
22                 print("Cannot access private repository without authentication.")
23                 return
24
25         if response.status_code == 403:
26             print("Rate limit exceeded. Please try again later or use authentication.")
27             return
28
29         if response.status_code == 200:
30             data = response.json()
31             print(f"\n{'='*60}")
32             print(f"Repository: {data.get('full_name', 'N/A')}")
33             print(f"{'='*60}")
34             print(f>Description: {data.get('description', 'No description')}")
35             print(f>Stars: {data.get('stargazers_count', 0):,}")
36             print(f>Forks: {data.get('forks_count', 0):,}")
37             print(f>Open Issues: {data.get('open issues count', 0):,}")
```



```

github_repo_info.py > ...
3  def fetch_repo_info():
22      print("Cannot access private repository without authentication.")
23      return
24
25      if response.status_code == 403:
26          print("Rate limit exceeded. Please try again later or use authentication.")
27          return
28
29      if response.status_code == 200:
30          data = response.json()
31          print(f"\n{'='*60}")
32          print(f"Repository: {data.get('full_name', 'N/A')}")
33          print(f"{'='*60}")
34          print(f>Description: {data.get('description', 'No description')}")
35          print(f"Stars: {data.get('stargazers_count', 0):,}")
36          print(f>Forks: {data.get('forks_count', 0):,}")
37          print(f>Open Issues: {data.get('open_issues_count', 0):,}")
38          print(f"{'='*60}")
39      else:
40          print(f>Error: {response.status_code} - {response.text}")
41  except Exception as e:
42      print(f>Error: {e}")
43
44  if __name__ == "__main__":
45      fetch_repo_info()
46

```

Output:

```

C:\Users\nadhi\OneDrive\Desktop\AI> python github_repo_info.py
Enter repository name (owner/repo, e.g., NadhiyaSeelam/AI_ASSISTED_CODING_2.1): NadhiyaSeelam/AI_ASSISTED_CODING_2.1

Attempting to fetch repository info...

C:\Users\nadhi\OneDrive\Desktop\AI> python github_repo_info.py
Enter repository name (owner/repo, e.g., NadhiyaSeelam/AI_ASSISTED_CODING_2.1): NadhiyaSeelam/AI_ASSISTED_CODING_2.1

Attempting to fetch repository info...

ISTED_CODING_2.1

Attempting to fetch repository info...

=====
Repository: NadhiyaSeelam/AI_ASSISTED_CODING_2.1
=====
Attempting to fetch repository info...

=====
Repository: NadhiyaSeelam/AI_ASSISTED_CODING_2.1
Description: None
Stars: 0
=====
Repository: NadhiyaSeelam/AI_ASSISTED_CODING_2.1
Description: None
Stars: 0
Description: None
Stars: 0
Forks: 0
Open Issues: 0
=====

C:\Users\nadhi\OneDrive\Desktop\AI>

```

Observation:

The code effectively retrieves and displays GitHub repository details with clear handling for authentication, rate limits, and missing repositories.

It's concise, user-friendly, and adaptable for both public and private repos.

Task -04

Prompt:

Generate and automatically run a **Python script (under 50 lines)** that uses a **free News API** (like [NewsAPI.org](https://newsapi.org/)) to fetch and display the **top 5 headlines** for a given news category such as sports, technology, or health.

The script should:

- Use the **requests** library to make API calls.
- Prompt the user to enter a news category.
- Handle errors for invalid category, missing or invalid API key, and network failures.
- Include a simple **retry mechanism** (try again once if the first request fails).
- Display the top 5 headlines in a **numbered list** format for easy reading.
- Keep the code **simple, readable, and under 50 lines**.
- **Automatically execute** after generation to show live results.

Also include:

1. **Three assert test cases** to verify correct data format and error handling.
2. **Inline comments** explaining key parts of the code.
3. Brief analysis of whether all tests pass.

Code generated:

```

1 import sys, requests
2
3 CATEGORIES = {"sports": "sport", "technology": "technology", "health": "health", "business": "business", "world": "world"}
4
5 # Validate category and map to Guardian section
6 def validate_category(category: str) -> str:
7     if not isinstance(category, str):
8         raise ValueError("Category must be a string")
9     key = category.strip().lower()
10    if key not in CATEGORIES:
11        raise ValueError(f"Invalid category: {category}")
12    return CATEGORIES[key]
13
14 # Build API URL using free Guardian Open Platform test key
15 def build_url(section: str) -> str:
16     return f"https://content.guardianapis.com/search?section={section}&page-size=5&order-by=newest&api-key=tes"
17
18 # Extract titles from API JSON
19 def parse_titles(data: dict) -> list:
20     results = (data or {}).get("response", {}).get("results", [])
21     return [it.get("webTitle", "").strip() for it in results if it.get("webTitle")][:5]
22
23 # Fetch headlines with one retry on failure
24 def fetch_headlines(category: str) -> list:
25     section = validate_category(category)
26     url = build_url(section)
27     for attempt in range(2):
28         try:
29             r = requests.get(url, timeout=8)
30             r.raise_for_status()
31             return parse_titles(r.json())
32         except Exception:
33             if attempt == 1:

```

```

24 def fetch_headlines(category: str) -> list:
32     except Exception:
33         if attempt == 1:
34             raise
35     return []
36
37 def main():
38     if sys.stdin.isatty():
39         try:
40             category = input("Enter news category (sports/technology/health/business/world): ")
41         except EOFError:
42             category = "technology"
43     else:
44         category = "technology" # default for non-interactive run
45     try:
46         titles = fetch_headlines(category)
47         if not titles:
48             print("No articles found.")
49         else:
50             for i, t in enumerate[Any](titles, 1):
51                 print(f"{i}. {t}")
52     except ValueError as e:
53         print(f"Error: {e}")
54     except requests.exceptions.RequestException as e:
55         print(f"Network/API error: {e}")
56
57 # ---- Simple tests ----
58 assert validate_category("technology") == "technology"
59 assert parse_titles([{"response": {"results": [{"webTitle": "A"}, {"webTitle": "B"}]}}]) == ["A", "B"]
60 try:
61     validate_category("invalid_cat")
62     _raised = False

```

```

news_headlines.py > ...
37 def main():
38     """
39     """
40     print("No articles found.")
41     else:
42         for i, t in enumerate[Any](titles, 1):
43             print(f"{i}. {t}")
44     except ValueError as e:
45         print(f"Error: {e}")
46     except requests.exceptions.RequestException as e:
47         print(f"Network/API error: {e}")
48
49 # ---- Simple tests ----
50 assert validate_category("technology") == "technology"
51 assert parse_titles({"response": {"results": [{"webTitle": "A"}, {"webTitle": "B"}]}}) == ["A", "B"]
52 try:
53     validate_category("invalid_cat")
54     _raised = False
55 except ValueError:
56     _raised = True
57 assert _raised, "Expected ValueError for invalid category"
58
59 if __name__ == "__main__":
60     main()
61
62
63
64
65
66
67
68
69
70

```

Output:

```

C:\Users\nadhi\OneDrive\Desktop\AI>python news_headlines.py
Enter news category (sports/technology/health/business/world): sports
1. The Chiefs-Bills rivalry is incapable of producing a bad game
2. Talking Horses | Willie Mullins eyes Melbourne and Breeders' Cup double after shock triumph
3. Eli Katoa's seizure and brain bleed prompts calls for NRL to assess concussion protocols
4. NFL week nine: Bills take down Chiefs, Jaguars edge Raiders in OT thriller - as it happened
5. Pitch invading prankster Daniel Jarvis charged after lining up with Kangaroos players at Ashes Test
C:\Users\nadhi\OneDrive\Desktop\AI>

```

Observation:

The script successfully fetched and displayed sports news headlines from various sources. It demonstrates effective use of categorized data retrieval and command-line output formatting.

